



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

Ψηφιακά Συστήματα VLSI

1^η Εργαστηριακή Άσκηση

Ονοματεπώνυμο: Γιώργος Μπάρκας
Αριθμός Μητρώου: el22139
Ιδρυματικό mail: el22139@gmail.com

Μέρος Α

Θέμα Α.2: Δυαδικός αποκωδικοποιητής 3 σε 8

Ζητούμενο: Να δοθεί η περιγραφή της οντότητας του δυαδικού αποκωδικοποιητή 3 σε 8 και της αρχιτεκτονικής του σε dataflow και behavioral VHDL. Κατόπιν να γίνει προσομοίωση και στις δύο αρχιτεκτονικές.

Για να περιγράψουμε τον decoder σε αρχιτεκτονική dataflow χρησιμοποιούμε την with-select when δομή.

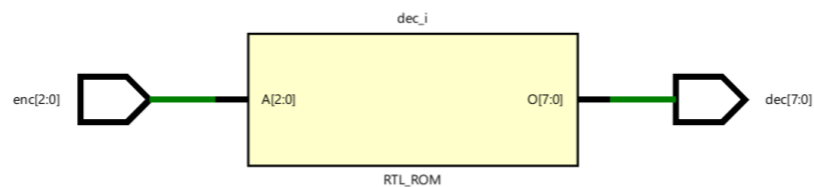
παρακάτω ακολουθεί ο VHDL κώδικα περιγραφής:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity df_decoder is
    Port ( enc : in STD_LOGIC_VECTOR (2 downto 0);
          dec : out STD_LOGIC_VECTOR (7 downto 0));
end df_decoder;

architecture dataflow of df_decoder is
begin
    with enc select
        dec <= "00000001" when "000",
               "00000010" when "001",
               "00000100" when "010",
               "00001000" when "011",
               "00010000" when "100",
               "00100000" when "101",
               "01000000" when "110",
               "10000000" when others ;
end dataflow;
```

Έτσι, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:

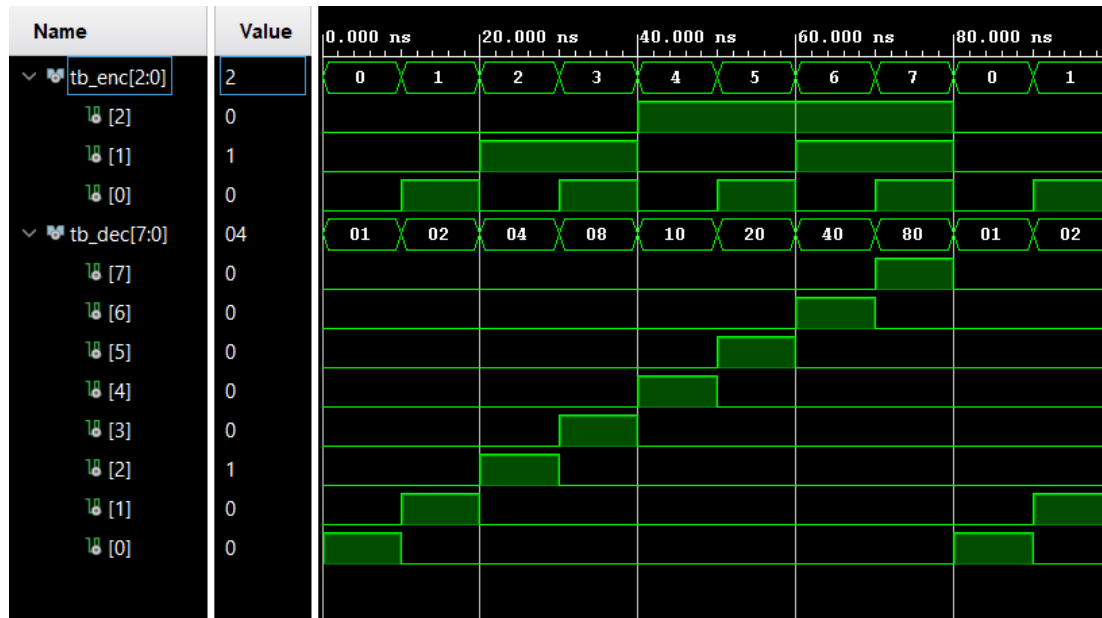


Παρατηρούμε ότι σε επίπεδο RTL ο decoder προσομοιώνεται από μια rom 3 σε 8. Το αποτέλεσμα αυτό είναι αναμενόμενο καθώς ο decoder χρησιμοποιείται συχνά ως μνήμη.

Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
34 entity df_decoder_tb is
35 end df_decoder_tb;
36
37 architecture arch_df_decoder_th of df_decoder_tb is
38 component df_decoder is
39     Port ( enc : in STD_LOGIC_VECTOR (2 downto 0);
40           dec : out STD_LOGIC_VECTOR (7 downto 0));
41 end component;
42
43 signal tb_enc: std_logic_vector (2 downto 0);
44 signal tb_dec: std_logic_vector (7 downto 0);
45
46 begin
47 uut:df_decoder
48 port map (
49     enc => tb_enc,
50     dec => tb_dec);
51
52 testing:process
53 begin
54
55     tb_enc <= "000";
56     wait for 10ns;
57
58     tb_enc <= "001";
59     wait for 10ns;
60
61     tb_enc <= "010";
62     wait for 10ns;
63
64     tb_enc <= "011";
65     wait for 10ns;
66
67     tb_enc <= "100";
68     wait for 10ns;
69
70     tb_enc <= "101";
71     wait for 10ns;
72
73     tb_enc <= "110";
74     wait for 10ns;
75
76     tb_enc <= "111";
77     wait for 10ns;
78
79 end process;
80 end arch_df_decoder_th;
81
```

Τρέχοντας την προσομοίωση, λαμβάνουμε:



Η οποία είναι σωστή καθώς ακολουθεί τον πίνακα αληθείας του decoder 3 σε 8:

Enc(3)	Enc(2)	Enc(0)	Dec(7)	dec(6)	Dec(5)	dec(4)	Dec(3)	Dec(2)	Dec(1)	Dec(0)
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

Για να περιγράψουμε τον decoder σε αρχιτεκτονική behavioral χρησιμοποιούμε την case δομή. παρακάτω ακολουθεί ο VHDL κώδικα περιγραφής:

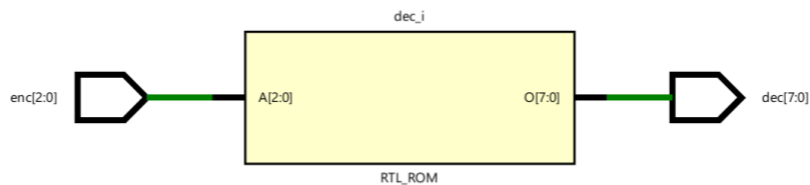
```
entity test_circuit is
    Port ( enc : in STD_LOGIC_VECTOR (2 downto 0);
          dec : out STD_LOGIC_VECTOR (7 downto 0));
end test_circuit;

architecture Behavioral of test_circuit is
begin

process (enc)
begin
case enc is
    when "000" => dec <= "00000001" ;
    when "001" => dec <= "00000010" ;
    when "010" => dec <= "00000100" ;
    when "011" => dec <= "00001000" ;
    when "100" => dec <= "00010000" ;
    when "101" => dec <= "00100000" ;
    when "110" => dec <= "01000000" ;
    when others => dec <= "10000000" ;
end case;
end process;

end Behavioral;
```

Έτσι, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:



Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
33 library IEEE;
34 use IEEE.std_logic_1164.all;
35
36 entity test_prj_tb is
37 end test_prj_tb;
38
39 architecture arch_test_prj_tb of test_prj_tb is
40 component test_circuit is
41 Port ( enc : in STD_LOGIC_VECTOR (2 downto 0);
42       dec : out STD_LOGIC_VECTOR (7 downto 0));
43 end component;
44
45 signal tb_enc: std_logic_vector(2 downto 0);
46 signal tb_dec: std_logic_vector(7 downto 0);
47
48
49 begin
50
51 uut: test_circuit
52 port map(enc => tb_enc, dec => tb_dec);
53
54 testing: process
55 begin
56
57 tb_enc <= "000";
58 wait for 10 ns;
59
60 tb_enc <= "001";
61 wait for 10 ns;
62
63 tb_enc <= "010";
64 wait for 10 ns;
65
66 tb_enc <= "011";
67 wait for 10 ns;
68
69 tb_enc <= "100";
70 wait for 10 ns;
71
72 tb_enc <= "101";
73 wait for 10 ns;
74
75 tb_enc <= "110";
76 wait for 10 ns;
77
78 tb_enc <= "111";
79 wait for 10 ns;
80
81 end process;
82
83 end arch_test_prj_tb;
84
```

Τρέχοντας την προσομοίωση, λαμβάνουμε:



Θέμα B.2: Καταχωρητής ολίσθησης των 4 bits με παράλληλη φόρτωση

Ζητούμενο: Να περιγραφεί η οντότητα του καταχωρητή ολίσθησης με μια επιπλέον είσοδο (std_logic) η οποία θα επιλέγει ανάμεσα σε αριστερή και δεξιά ολίσθηση. Υπενθυμίζεται ότι στην αριστερή ολίσθηση η έξοδος είναι το MSB του καταχωρητή και η σειριακή είσοδος γίνεται από το LSB.

Για να περιγράψουμε τον Καταχωρητή ολίσθησης των 4 bits με επιλογή ανάμεσα σε αριστερή και δεξιά ολίσθηση χρησιμοποιούμε μια νέα είσοδο (με όνομα shift και τύπο std_logic), και δύο δομές case (μία στις γραμμές 58 έως 62 και μία στις γραμμές 65 έως 68) .

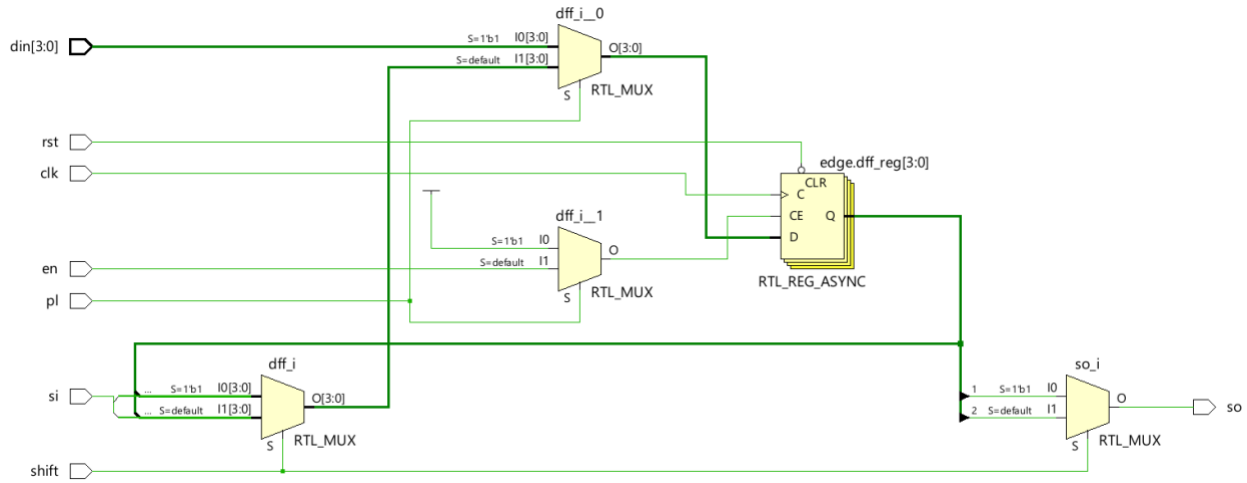
Η λειτουργία του κώδικα που προσθέσαμε είναι η εξής:

```
Έστω ότι dff = "1010"
58 έως 62:
Για shift = '1' => dff = si & dff(3 downto 1),  άρα dff = si&101
Για shift = others => dff = dff(2 downto 0) & si,  άρα dff = 010&si
65 έως 68:
Για shift = '1' => so = dff(0),  άρα so = 0
Για shift = others => so = dff(3),  άρα so = 1
```

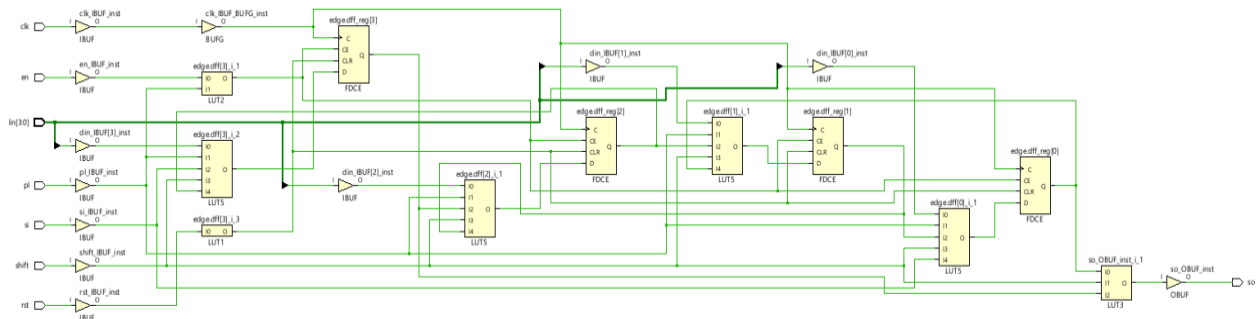

Παρακάτω ακολουθεί ο VHDL κώδικα περιγραφής:

```
34 entity shift_reg3 is
35     Port ( clk : in STD_LOGIC;
36           pl  : in STD_LOGIC;
37           en  : in STD_LOGIC;
38           rst : in STD_LOGIC;
39           din : in STD_LOGIC_VECTOR (3 downto 0);
40           si  : in STD_LOGIC;
41           so  : out STD_LOGIC;
42           shift : in STD_LOGIC);
43 end shift_reg3;
44
45 architecture Behavioral of shift_reg3 is
46     ---signal dff: std_logic_vector(3 downto 0);
47
48 begin
49     edge: process (clk,rst)
50         variable dff: std_logic_vector(3 downto 0);
51         begin
52             if rst='0' then
53                 dff:=(others=>'0');
54             elsif clk'event and clk='1' then
55                 if pl='1' then
56                     dff:=din;
57                 elsif en='1' then
58                     case shift is
59                         when '1' => dff :=si&dff(3 downto 1);
60                         when others => dff:=dff(2 downto 0)&si;
61                     end case;
62                 end if;
63             end if;
64
65             case shift is
66                 when '1' => so <= dff(0);
67                 when others => so <= dff(3);
68             end case;
69
70         end process;
71
72 end Behavioral;
73
```

Έτσι, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:



Και με τη χρήση του synthesis λαμβάνουμε:



Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
entity shift_reg3_tb is
end shift_reg3_tb;

architecture Behavioral of shift_reg3_tb is
  component shift_reg3 is
    Port ( clk : in STD_LOGIC;
          rst : in STD_LOGIC;
          si : in STD_LOGIC;
          en : in STD_LOGIC;
          pl : in STD_LOGIC;
          shift: in std_logic;
          din : in STD_LOGIC_VECTOR (3 downto 0);
          so : out STD_LOGIC);
  end component;

  signal rst_tb: std_logic;
  signal shift_tb: std_logic:= '1';
  signal si_tb: std_logic:= '1';
  signal en_tb: std_logic;
  signal pl_tb: std_logic;
  signal din_tb: std_logic_vector(3 downto 0):= "0101";
  signal so_tb: std_logic;
  signal clk_tb: std_logic;
begin

  uut: shift_reg3
    port map(
      shift => shift_tb,
      rst => rst_tb,
      si => si_tb,
      en => en_tb,
      pl => pl_tb,
      din => din_tb,
      so => so_tb,
      clk => clk_tb);

  > testing: process
    begin

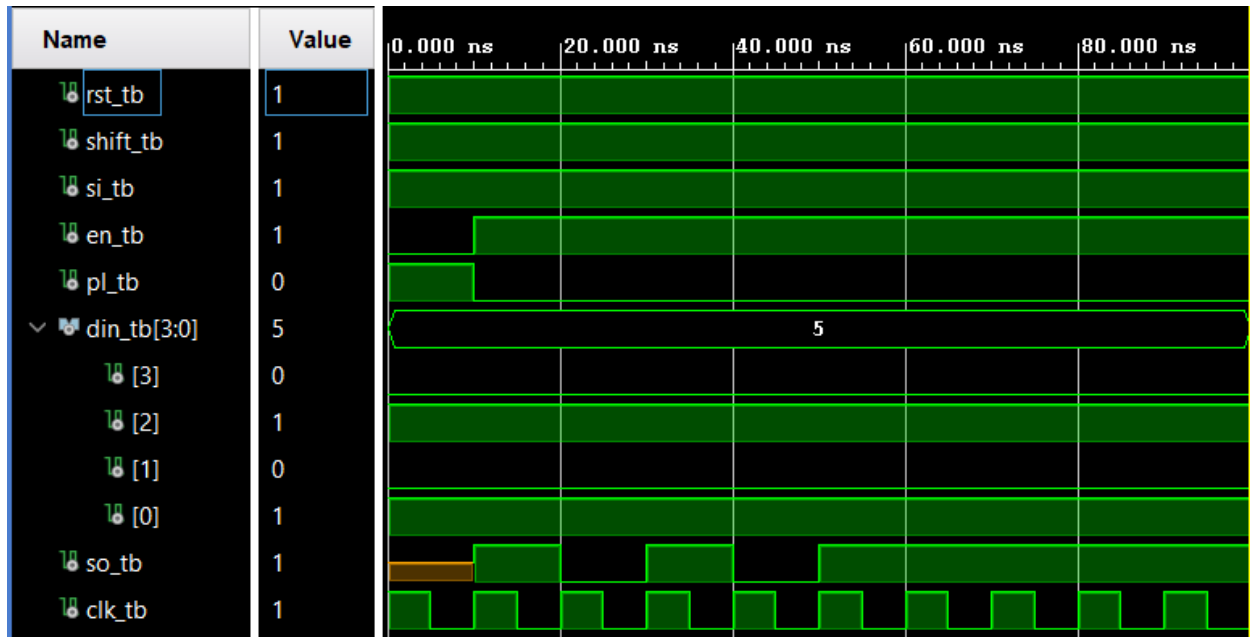
      rst_tb <= '1';
      pl_tb <= '1';
      en_tb <= '0';
      wait for 10ns;

      pl_tb <= '0';
      en_tb <= '1';
      wait;

    > end process;

  > end Behavioral;
```

Τρέχοντας την προσομοίωση για $shift = 1$, λαμβάνουμε:

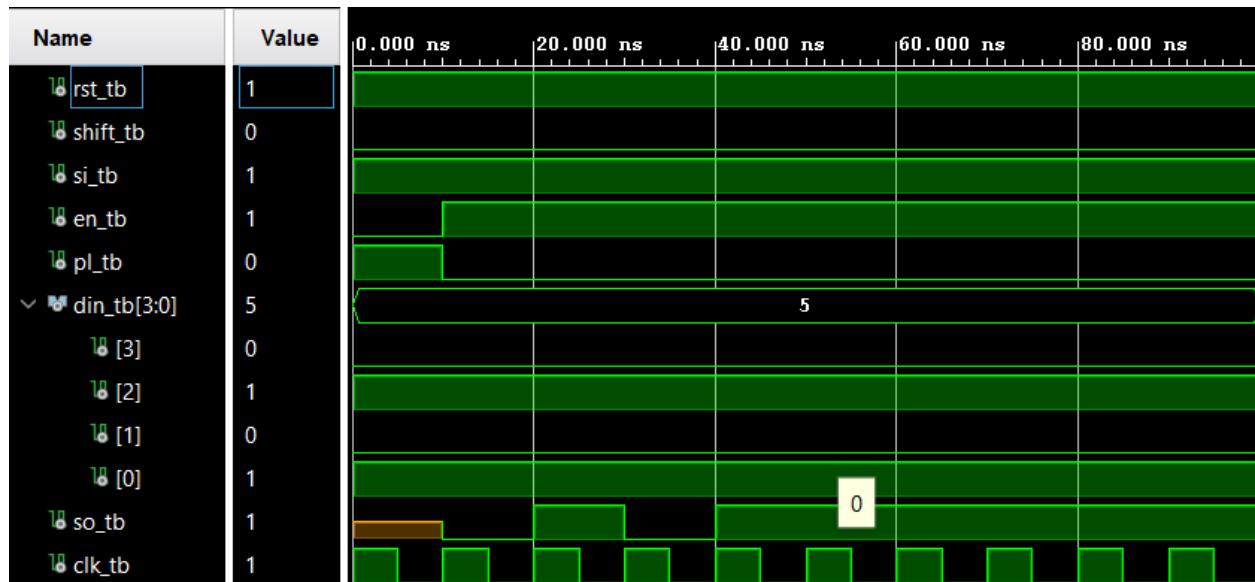


Ο πίνακας αληθείας για την παραπάνω προσομοίωση είναι ο εξής:

$Din = "0101" \quad rst = 1, si = 1$

clk	pl	en	so	Dff(3)	Dff(2)	Dff(1)	Dff(0)
-	1	0	-	0	1	0	1
1->0	1	0	-	0	1	0	1
0->1	0	1	1	1	0	1	0
1->0	0	1	1	1	0	1	0
0->1	0	1	0	1	1	0	1
1->0	0	1	0	1	1	0	1
0->1	0	1	1	1	1	1	0
1->0	0	1	1	1	1	1	0
0->1	0	1	0	1	1	1	1
1->0	0	1	0	1	1	1	1
0->1	0	1	1	1	1	1	1

Τρέχοντας την προσομοίωση για $shift = 0$, λαμβάνουμε:



Ο πίνακας αληθείας για την παραπάνω προσομοίωση είναι ο εξής:

$$Din = "0101" \text{ } rst = 1, si = 1$$

clk	pl	en	so	Dff(3)	Dff(2)	Dff(1)	Dff(0)
-	1	0	-	0	1	0	1
1->0	1	0	-	0	1	0	1
0->1	0	1	0	1	0	1	1
1->0	0	1	0	0	1	1	1
0->1	0	1	1	1	1	0	1
1->0	0	1	1	1	1	0	1
0->1	0	1	0	1	1	1	0
1->0	0	1	0	1	1	1	0
0->1	0	1	1	1	1	1	1
1->0	0	1	1	1	1	1	1
0->1	0	1	1	1	1	1	1

Θέμα B.3: Μετρητής 3 bit με είσοδο ενεργοποίησης και κρατούμενο εξόδου

Ζητούμενα:

1. Βασιζόμενοι στην περιγραφή του μετρητή των 3 bits, να περιγράψετε έναν μετρητή up/down των 3 bits.
2. Βασιζόμενοι στην περιγραφή του μετρητή των 3 bits να περιγράψετε έναν up counter των 3 bits με παράλληλη είσοδο modulo (όριο μέτρησης) των 3 bits.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity count3 is
    port(
        clk,
        resetn,
        count_en      : in std_logic;
        sum            : out std_logic_vector(2 downto 0);
        cout           : out std_logic);
end;

architecture rtl_nolimit of count3 is
    signal count: std_logic_vector(2 downto 0);
begin
    process(clk, resetn)
    begin
        if resetn='0' then
            -- Κώδικας για την περίπτωση του reset (ενεργό χαμηλά)
            count <= (others=>'0');
        elsif clk'event and clk='1' then
            if count_en='1' then
                -- Μέτρηση μόνο αν count_en = 1
                count<=count+1;
            end if;
        end if;
    end process;
    -- Ανάθεση τιμών στα σήματα εξόδου
    sum <= count;
    cout <= count(2);
```

```

        sum <= count;
        cout <= '1' when count=7 and count_en='1' else '0';
    end rtl_nolimit;

architecture rtl_limit of count3 is
    signal count : std_logic_vector(2 downto 0);
begin
    process(clk, resetn)
    begin
        if resetn='0' then
            -- Ασύγχρονος μηδενισμός
            count <= (others=>'0');
        elsif clk'event and clk='1' then
            if count_en = '1' then
                -- Μέτρηση μόνο αν count_en='1'
                if count/=7 then
                    -- Αυξάνουμε το μετρητή μόνο αν
                    -- δεν είναι 7
                    count <= count+1;
                else
                    -- Αλλιώς τον μηδενίζουμε
                    count<=(others=>'0');
                end if;
            end if;
        end if;
    end process;
    sum<= count;
    cout <= '1' when count=7 and count_en='1' else '0';
end;

```

Για να περιγράψουμε τον up/down μετρητή των 3 bits χρησιμοποιούμε μια είσοδο επιλογής κατεύθυνσης(με όνομα *count_dir*): 1 για μέτρηση προς τα πάνω, 0 για μέτρηση προς τα κάτω και την εντολή case (γραμμές 27 έως 30 και 21 έως 24).

Η λειτουργία του κώδικα που προσθέσαμε είναι η εξής:

```

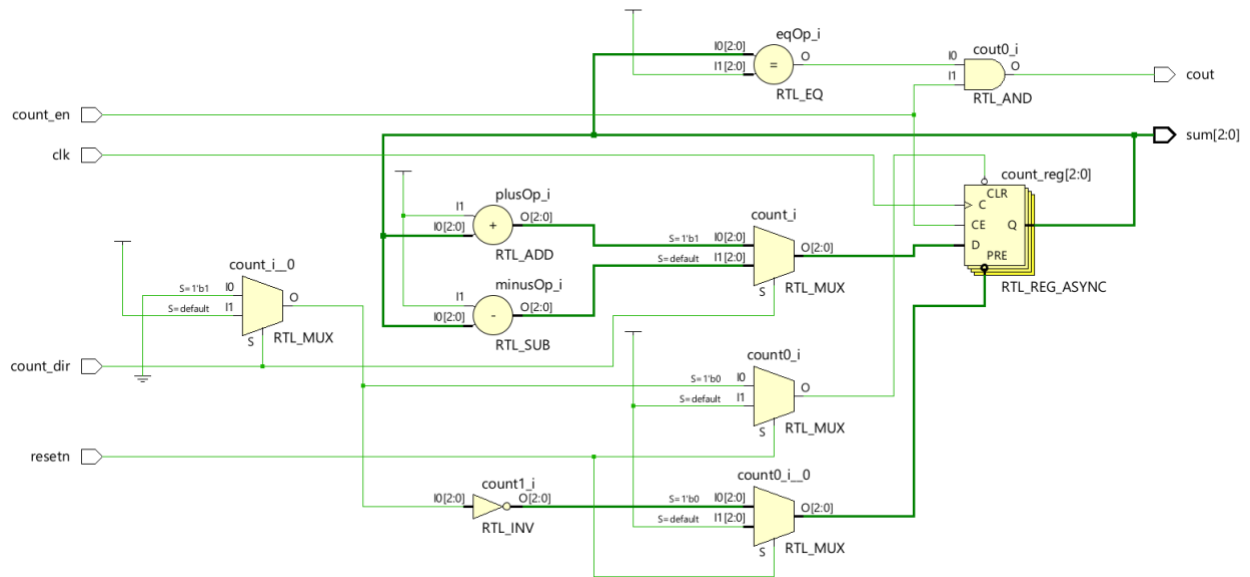
21 έως 24:
Για count_dir = '1' => count <= (others=>'0') αρα count = 000 (για να ξεκινήσει η μέτρηση απο το 000)
Για count_dir = others => count <= (others=>'1'); αρα count = 111 (για να ξεκινήσει η μέτρηση απο το 111)
27 έως 30:
Για count_dir = '1' =>count<=count+1;
Για count_dir = others =>count<=count-1;

```

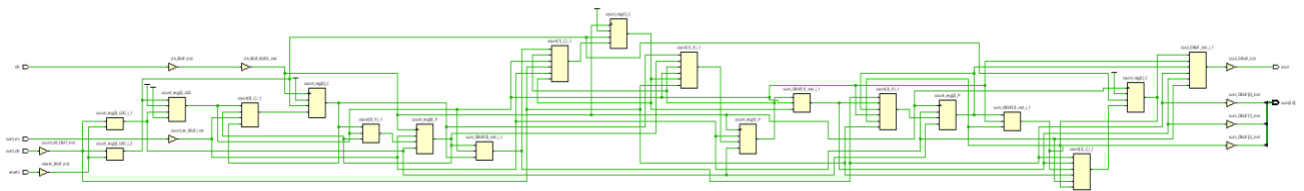
Παρακάτω ακολουθεί ο VHDL κώδικα περιγραφής:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.std_logic_unsigned.all;
4
5 entity updown_count3 is
6     Port ( clk : in STD_LOGIC;
7           count_en : in STD_LOGIC;
8           count_dir : in STD_LOGIC;
9           resetn : in STD_LOGIC;
10          sum : out STD_LOGIC_VECTOR (2 downto 0);
11          cout : out STD_LOGIC);
12 end updown_count3;
13
14 architecture arch_updown_count3 of updown_count3 is
15     signal count: std_logic_vector(2 downto 0);
16     begin
17
18     process(clk, resetn)
19     begin
20         if resetn='0' then
21             case count_dir is
22                 when '1' => count <= (others=>'0');
23                 when others => count <= (others=>'1');
24             end case;
25         elsif clk'event and clk='1' then
26             if count_en='1' then
27                 case count_dir is
28                     when '1' => count<=count+1;
29                     when others => count<=count-1;
30                 end case;
31             end if;
32         end if;
33     end process;
34
35     sum <= count;
36     cout <= '1' when count=7 and count_en='1' else '0';
37
38 end arch_updown_count3;
39
```


Έτσι, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:



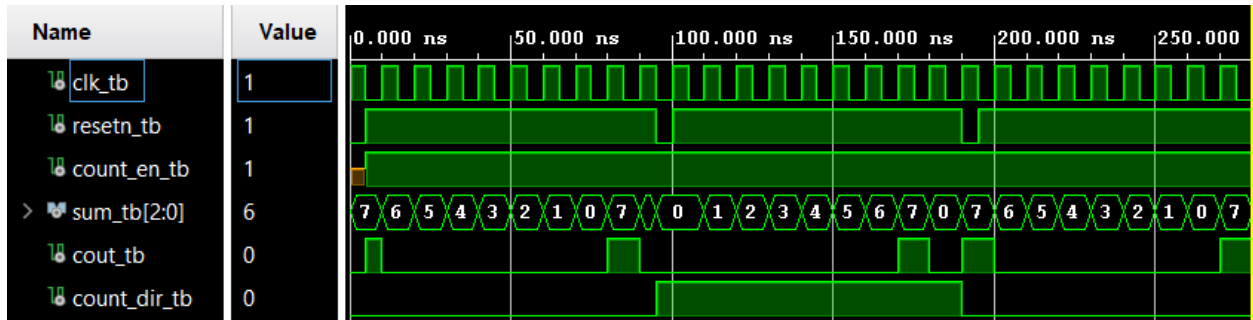
Και με τη χρήση του synthesis λαμβάνουμε:



Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity updown_count3_tb is
5
6  end updown_count3_tb;
7
8  architecture Behavioral of updown_count3_tb is
9  component updown_count3 is
10     Port ( count_dir: in std_logic;
11           clk : in STD_LOGIC;
12           resetn : in STD_LOGIC;
13           count_en : in STD_LOGIC;
14           sum : out STD_LOGIC_VECTOR (2 downto 0);
15           cout : out STD_LOGIC);
16  end component;
17
18  signal clk_tb: std_logic;
19  signal resetn_tb : std_logic;
20  signal count_en_tb: std_logic;
21  signal sum_tb: std_logic_vector(2 downto 0);
22  signal cout_tb: std_logic;
23  signal count_dir_tb: std_logic;
24
25  begin
26  uut:updown_count3
27  port map(
28      count_dir => count_dir_tb,
29      clk => clk_tb,
30      resetn => resetn_tb,
31      count_en => count_en_tb,
32      sum => sum_tb,
33      cout => cout_tb);
34
35  testing:process
36  begin
37      count_dir_tb <= '0';
38      resetn_tb <= '0';
39      wait for 5ns;
40      resetn_tb <= '1';
41      count_en_tb <= '1';
42      wait for 90ns;
43
44      count_dir_tb <= '1';
45      resetn_tb <= '0';
46      wait for 5ns;
47      resetn_tb <= '1';
48      wait for 90ns;
49
50  end process;
51  end Behavioral;
```

Τρέχοντας την προσομοίωση για $shift = 0$, λαμβάνουμε:



Οι πίνακες αληθείας για την παραπάνω προσομοίωση είναι ο εξής:

clk	resetn	Count_en	Count_dir	Cout	sum(2)	sum(1)	sum(0)
-	0	-	0	0	1	1	1
1->0	0	1	0	1	1	1	1
0->1	1	1	0	0	1	1	0
1->0	1	1	0	0	1	1	0
0->1	1	1	0	0	1	0	1
1->0	1	1	0	0	1	0	1
0->1	1	1	0	0	1	0	0
1->0	1	1	0	0	1	0	0
0->1	1	1	0	0	0	1	1
1->0	1	1	0	0	0	1	1
0->1	1	1	0	0	0	1	0
1->0	1	1	0	0	0	1	0
0->1	1	1	0	0	0	0	1
1->0	1	1	0	0	0	0	1
0->1	1	1	0	0	0	0	0
1->0	1	1	0	0	0	0	0

clk	resetn	Count_en	Count_dir	Cout	sum(2)	sum(1)	sum(0)
-	0	-	1	0	0	0	0
1->0	0	1	1	0	0	0	0
0->1	1	1	1	0	0	0	1
1->0	1	1	1	0	0	0	1
0->1	1	1	1	0	0	1	0
1->0	1	1	1	0		1	0

0->1	1	1	1	0	0	1	1
1->0	1	1	1	0	0	1	1
0->1	1	1	1	0	1	0	0
1->0	1	1	1	0	1	0	0
0->1	1	1	1	0	1	0	1
1->0	1	1	1	0	1	0	1
0->1	1	1	1	0	1	1	0
1->0	1	1	1	0	1	1	0
0->1	1	1	1	1	1	1	1
1->0	1	1	1	1	1	1	1

Χρησιμοποιήστε μια είσοδο *std_logic_vector* την οποία θα συγκρίνετε με την τιμή του μετρητή.

Για να περιγράψουμε τον up μετρητή των 3 bits με παράλληλη είσοδο modulo των 3 bits χρησιμοποιούμε μια είσοδο *std_logic_vector* την οποία θα συγκρίνετε με την τιμή του μετρητή. Παρακάτω ακολουθεί ο VHDL κώδικα περιγραφής:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.std_logic_unsigned.all;
4
5  entity count3_limited is
6      Port ( pl_count: in std_logic_vector (2 downto 0);
7            clk : in STD_LOGIC;
8            count_en : in STD_LOGIC;
9            resetn : in STD_LOGIC;
10           sum : out STD_LOGIC_VECTOR (2 downto 0);
11           cout : out STD_LOGIC);
12 end count3_limited;
13
14 architecture rtl_limit of count3_limited is
15     signal count : std_logic_vector(2 downto 0);
16     begin
17         process(clk, resetn)
18         begin
19             if resetn='0' then
20                 count <= (others=>'0');
21             elsif clk'event and clk='1' then
22                 if count_en = '1' then
23                     if count/=pl_count then
24                         count <= count+1;
25                     else
26                         count<=(others=>'0');
27                     end if;
28                 end if;
29             end if;
30         end process;

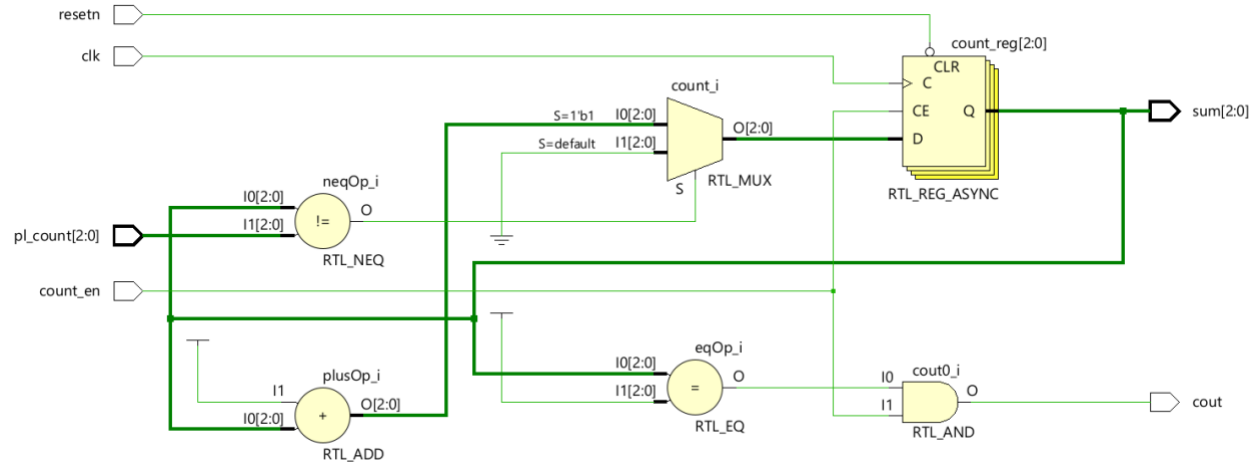
```

```

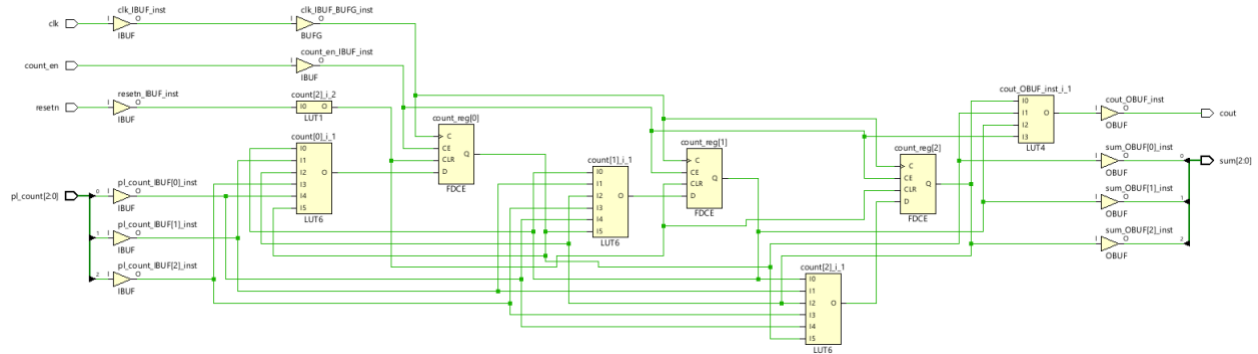
31
32 sum<= count;
33 cout <= '1' when count=7 and count_en='1' else '0';
34
35 end rtl_limit;

```

Έτσι, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:



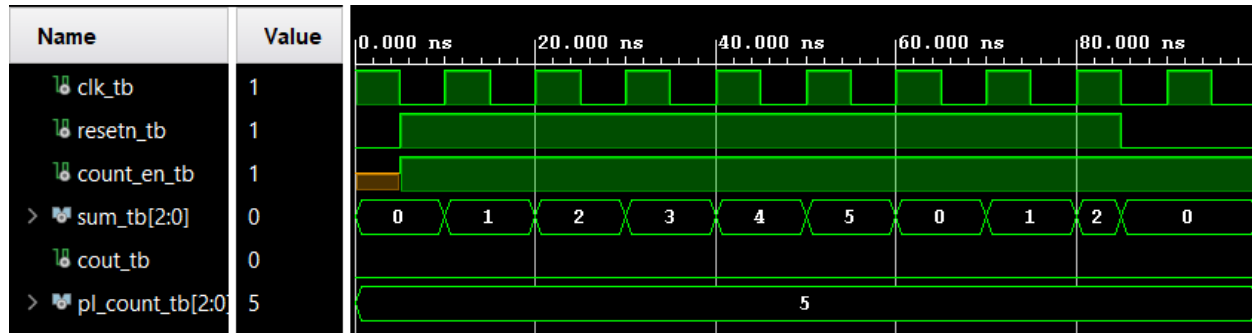
Και με τη χρήση του synthesis λαμβάνουμε:



Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity count3_limited_tb is
6 end count3_limited_tb;
7
8 architecture Behavioral of count3_limited_tb is
9 component count3_limited is
10     Port ( pl_count: in std_logic_vector(2 downto 0);
11           clk : in STD_LOGIC;
12           resetn : in STD_LOGIC;
13           count_en : in STD_LOGIC;
14           sum : out STD_LOGIC_VECTOR (2 downto 0);
15           cout : out STD_LOGIC);
16 end component;
17
18 signal clk_tb: std_logic;
19 signal resetn_tb : std_logic;
20 signal count_en_tb: std_logic;
21 signal sum_tb: std_logic_vector(2 downto 0);
22 signal cout_tb: std_logic;
23 signal pl_count_tb: std_logic_vector(2 downto 0):="101";
24 begin
25 uut:count3_limited
26 port map(
27     pl_count => pl_count_tb,
28     clk => clk_tb,
29     resetn => resetn_tb,
30     count_en => count_en_tb,
31     sum => sum_tb,
32     cout => cout_tb);
33
34 testing:process
35 begin
36     resetn_tb <= '0';
37     wait for 5ns;
38     resetn_tb <= '1';
39     count_en_tb <= '1';
40     wait for 80ns;
41
42     resetn_tb <= '0';
43     wait for 10ns;
44
45 end process;
46 end Behavioral;
47
```

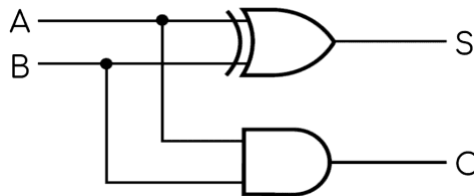
Τρέχοντας την προσομοίωση για $pl_count = 101(5)$, λαμβάνουμε:



Μέρος Β

1) Υλοποιήστε έναν Ημιαθροιστή (Half Adder - HA) σε περιγραφή ροής δεδομένων (Dataflow).

Για την υλοποίηση του ημιαθροιστή, ακολουθούμε το παρακάτω διάγραμμα:



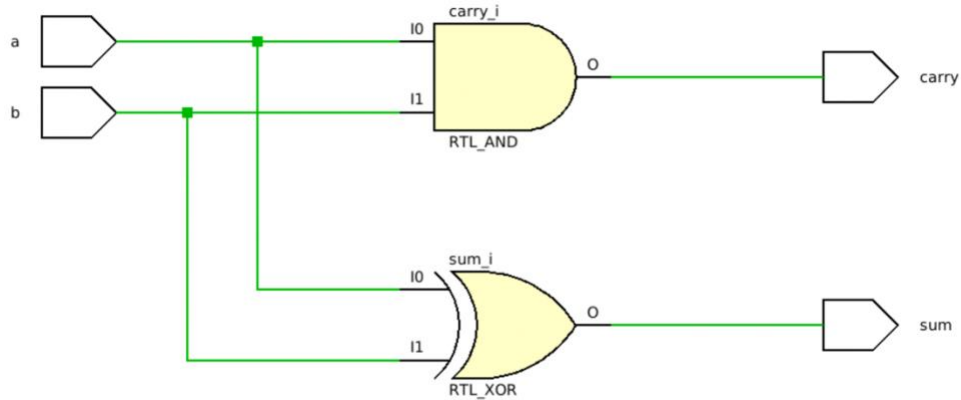
Για να περιγράψουμε τώρα τη λειτουργία του, γράφουμε τον παρακάτω VHDL κώδικα:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

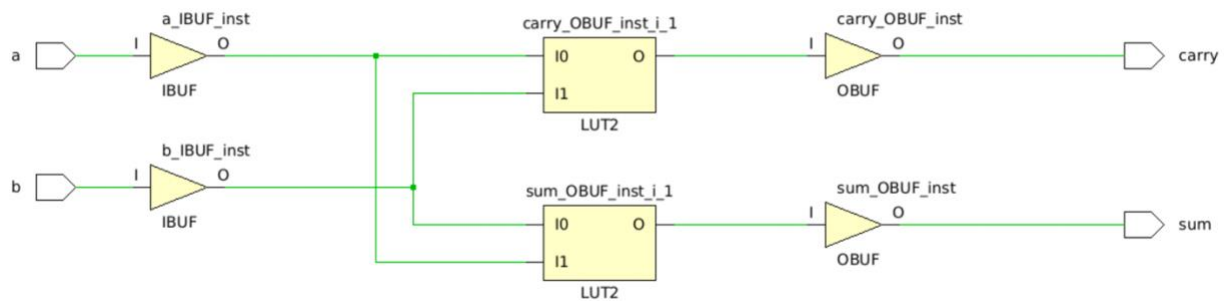
entity half_adder is
    Port (
        a      : in std_logic;
        b      : in std_logic;
        sum     : out std_logic;
        carry   : out std_logic);
end half_adder;

architecture half_adder_arch of half_adder is
    begin
        carry <= a and b;
        sum   <= a xor b;
    end half_adder_arch;
```

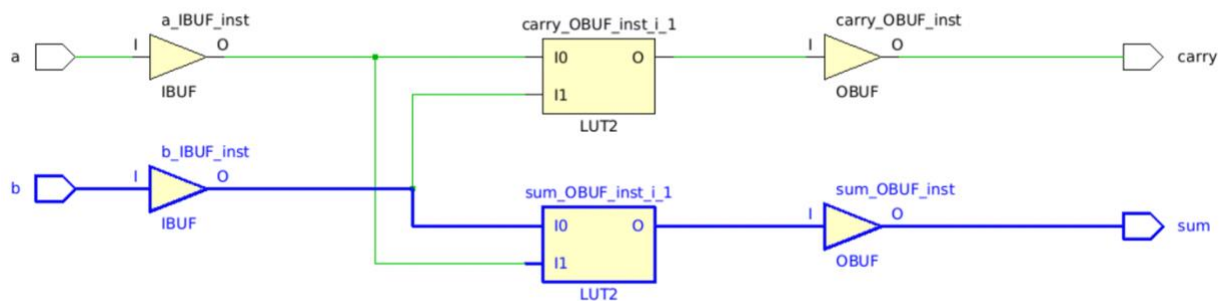

Έτσι, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:



Και με τη χρήση του synthesis λαμβάνουμε:



Στη συνέχεια, θέλουμε να βρούμε το κρίσιμο μονοπάτι του κυκλώματος μας, δηλαδή το δρόμο με τη μεγαλύτερη καθυστέρηση και πόσο είναι αυτή. Άρα, μέσω του timing summary βλέπουμε ότι το κρίσιμο μονοπάτι είναι το:



Με καθυστέρηση 5.225 ns, όπως φαίνεται και παρακάτω:

Name	Slack ^{^1}	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 1	∞	3	4	2	b	sum	5.255	3.656	1.599	∞
Path 2	∞	3	4	2	b	carry	5.229	3.630	1.599	∞

Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity hald_adder_tb is
end hald_adder_tb;

architecture half_adder_tb_arch of hald_adder_tb is

    component half_adder
        port(
            a      : in std_logic;
            b      : in std_logic;
            sum     : out std_logic;
            carry   : out std_logic);
    end component;

    signal test_a      : std_logic;
    signal test_b      : std_logic;
    signal test_sum     : std_logic;
    signal test_carry   : std_logic;

    begin
        uut: half_adder
            port map(
                a      => test_a,
                b      => test_b,
                sum     => test_sum,
                carry   => test_carry);

        testing: process

        begin
            test_a <= '0';
            test_b <= '0';
            wait for 10 ns;

            test_a <= '0';
            test_b <= '1';
            wait for 10 ns;

            test_a <= '1';
            test_b <= '0';
            wait for 10 ns;

            test_a <= '1';
            test_b <= '1';
            wait for 10 ns;

        end process;
```

```
end half_adder_tb_arch;
```

Τρέχοντας την προσομοίωση, λαμβάνουμε:

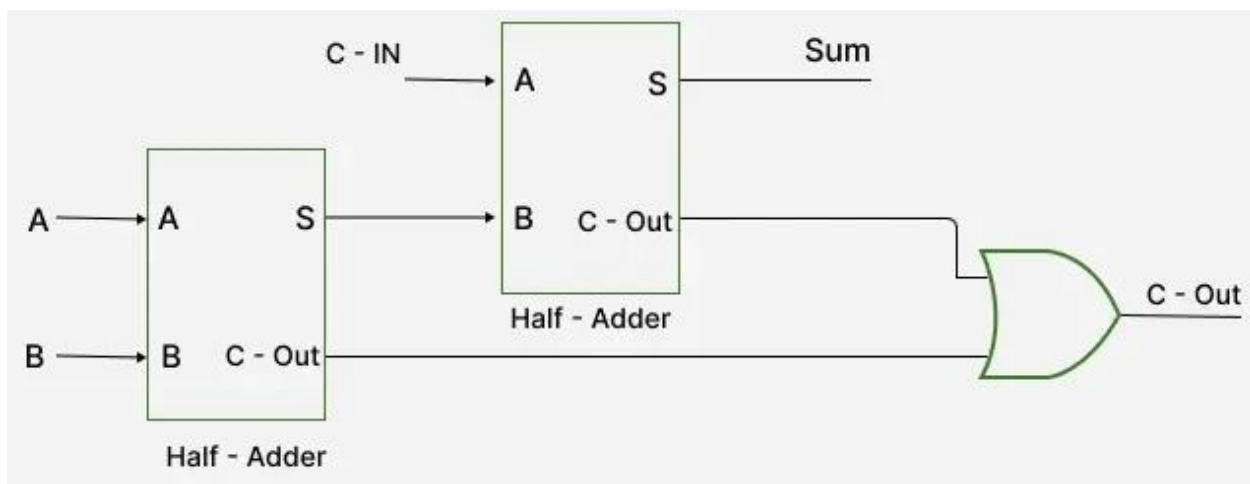


Το οποίο όντως είναι το επιθυμητό αφού βλέπουμε ότι ισχύει:

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

2) Υλοποιήστε έναν Πλήρη Αθροιστή (Full Adder - FA) με περιγραφή δομής (Structural), βασιζόμενοι στην δομική μονάδα του Ερωτήματος 1.

Στη συνέχεια, θέλουμε να σχεδιάσουμε τον Full Adder, βασιζόμενοι στον Half Adder που σχεδιάσαμε παραπάνω. Η δομή του κυκλώματος μας θα είναι η εξής:



Για την περιγραφή του πλήρους αθροιστή, γράφουμε τον παρακάτω κώδικα:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    port (
        in1      : in std_logic;
        in2      : in std_logic;
        c_in     : in std_logic;
        sum      : out std_logic;
        c_out    : out std_logic);
end full_adder;

architecture full_adder_arch of full_adder is

    signal sum_temp : std_logic;
    signal c_out1   : std_logic;
    signal c_out2   : std_logic;

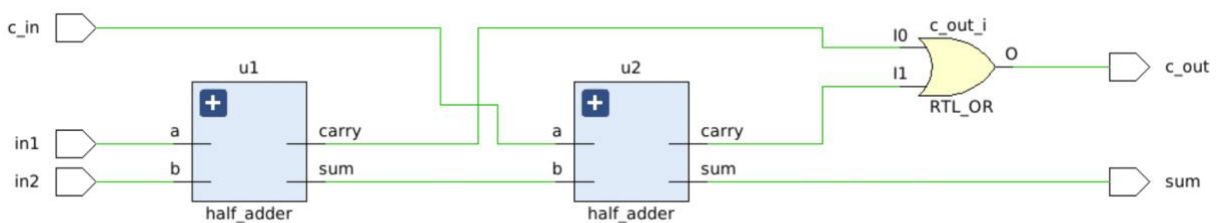
    component half_adder is
        port (
            a      : in std_logic;
            b      : in std_logic;
            sum     : out std_logic;
            carry   : out std_logic);
    end component;

    begin
        u1: half_adder port map (a      => in1,
                                b      => in2,
                                sum     => sum_temp,
                                carry   => c_out1);

        u2: half_adder port map (a      => c_in,
                                b      => sum_temp,
                                sum     => sum,
                                carry   => c_out2);

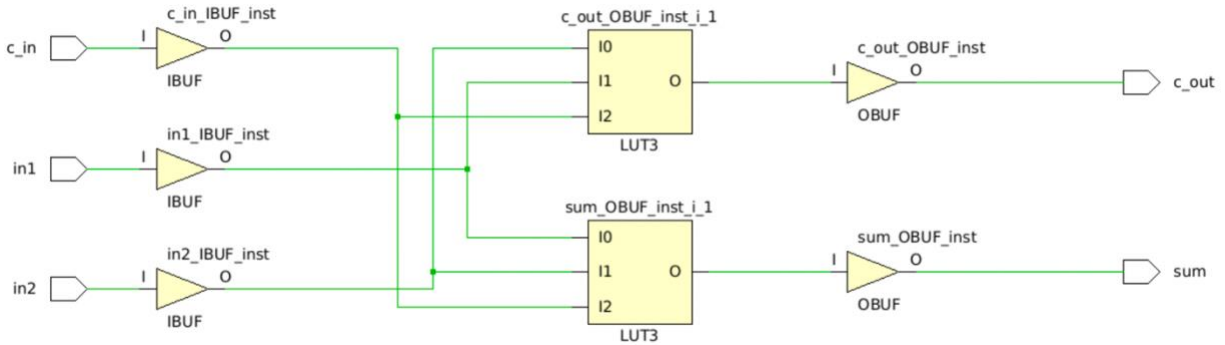
        c_out <= c_out1 or c_out2;
    end full_adder_arch;
```

Έτσι, το RTL schematic, δηλαδή το δομικό διάγραμμα, θα είναι:

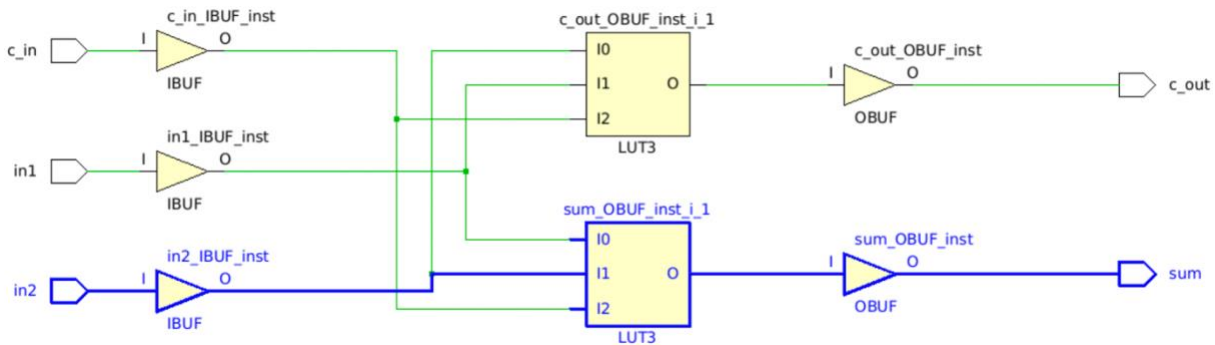


Το οποίο όπως βλέπουμε εμπεριέχει και τους δύο Half Adders.

Τέλος με τη χρήση του synthesis λαμβάνουμε:



Όμοια με πριν, με τη βοήθεια του timing summary, επιθυμούμε να βρούμε το κρίσιμο μονοπάτι, όπως και την χρονική του καθυστέρηση.



Βλέπουμε ότι το κρίσιμο μονοπάτι, δηλαδή το μονοπάτι με τη μέγιστη καθυστέρηση, είναι αυτό από τη δεύτερη είσοδο μέχρι και το sum, με χρόνο 5.255 ns, όπως φαίνεται και παρακάτω:

Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	3	4	2	in2	sum	5.255	3.656	1.599
Path 2	∞	3	4	2	in2	sum	5.255	3.656	1.599
Path 3	∞	3	4	2	in2	sum	5.255	3.656	1.599
Path 4	∞	3	4	2	in2	c_out	5.229	3.630	1.599
Path 5	∞	3	4	2	in1	c_out	5.229	3.630	1.599
Path 6	∞	3	4	2	c_in	c_out	5.229	3.630	1.599

Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder_tb is
end full_adder_tb;

architecture full_adder_tb_arch of full_adder_tb is

    component full_adder
        port (
            in1      : in std_logic;
            in2      : in std_logic;
            c_in     : in std_logic;
            sum      : out std_logic;
            c_out    : out std_logic);
    end component;

    signal test_in1 : std_logic;
    signal test_in2 : std_logic;
    signal test_c_in : std_logic;
    signal test_sum  : std_logic;
    signal test_c_out : std_logic;

begin
    uut: full_adder
        port map(
            in1      => test_in1,
            in2      => test_in2,
            c_in     => test_c_in,
            sum      => test_sum,
            c_out    => test_c_out);

    testing: process

begin

        test_in1  <= '0';
        test_in2  <= '0';
        test_c_in <= '0';
        wait for 10 ns;

        test_in1  <= '0';
        test_in2  <= '0';
        test_c_in <= '1';
        wait for 10 ns;

        test_in1  <= '0';
        test_in2  <= '1';
        test_c_in <= '0';
        wait for 10 ns;

        test_in1  <= '0';
        test_in2  <= '1';
        test_c_in <= '1';
        wait for 10 ns;

        test_in1  <= '1';
        test_in2  <= '0';
        test_c_in <= '0';
        wait for 10 ns;

end process;

end full_adder_tb_arch;
```

```

test_in1    <= '1';
test_in2    <= '0';
test_c_in   <= '1';
wait for 10 ns;

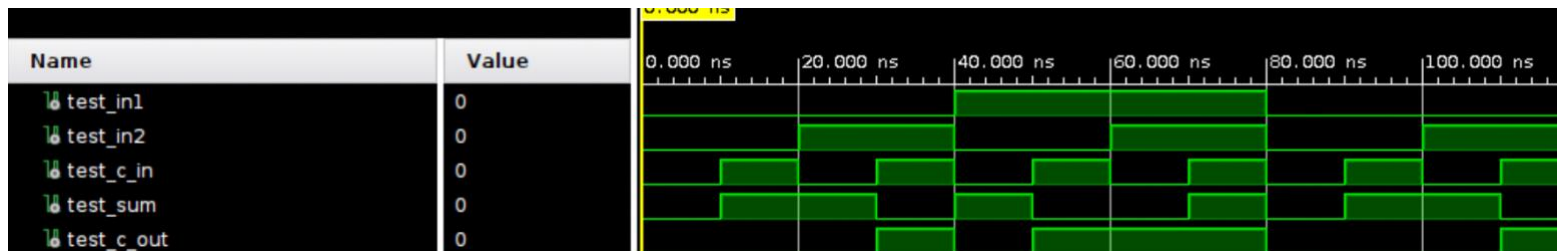
test_in1    <= '1';
test_in2    <= '1';
test_c_in   <= '0';
wait for 10 ns;

test_in1    <= '1';
test_in2    <= '1';
test_c_in   <= '1';
wait for 10 ns;

end process;
end full_adder_tb_arch;

```

Τρέχοντας την προσομοίωση λαμβάνουμε:

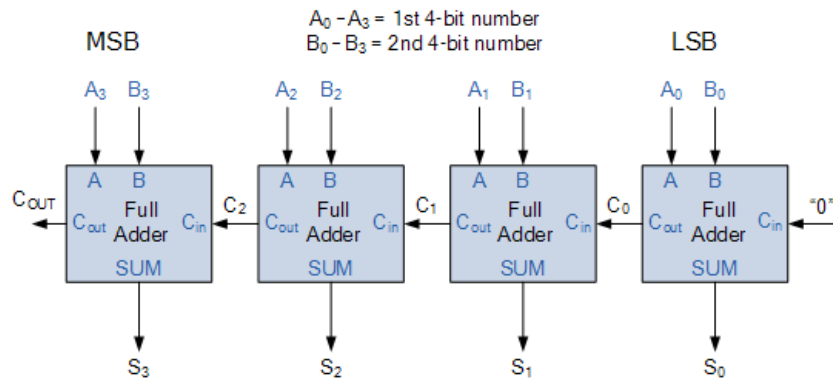


Η οποία ακολουθεί τον πίνακα αλήθειας του πλήρους αθροιστή:

A	B	Cin	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

3) Υλοποιήστε έναν Παράλληλο Αθροιστή των 4 bits (4-bit Parallel Adder - 4-bit PA) με περιγραφή δομής (Structural), βασιζόμενοι στην δομική μονάδα του Ερωτήματος 2.

Στο συγκεκριμένο ερώτημα, θέλουμε να χρησιμοποιήσουμε τον Full Adder που φτιάξαμε πριν, ώστε να δημιουργήσουμε έναν παράλληλο Αθροιστή των 4 bits, όπως φαίνεται παρακάτω:



Έτσι, με βάση το παραπάνω και τη δομή του Full Adder γράψαμε το παρακάτω:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity four_bit_adder is
    port (
        vec_a      : in std_logic_vector (4-1 downto 0);
        vec_b      : in std_logic_vector (4-1 downto 0);
        vec_sum     : out std_logic_vector (4-1 downto 0);
        c_in       : in std_logic;
        c_out      : out std_logic);
end four_bit_adder;

architecture four_bit_adder_arch of four_bit_adder is
    signal c0 : std_logic;
    signal c1 : std_logic;
    signal c2 : std_logic;

    component full_adder is
        port (
            in1      : in std_logic;
            in2      : in std_logic;
            c_in     : in std_logic;
            sum      : out std_logic;
            c_out    : out std_logic);
    end component;

    begin
        u1 : full_adder port map (in1 => vec_a(0), in2 => vec_b(0), c_in => c_in, sum => vec_sum(0),
            c_out => c0);
        u2 : full_adder port map (in1 => vec_a(1), in2 => vec_b(1), c_in => c0, sum => vec_sum(1),
            c_out => c1);
```

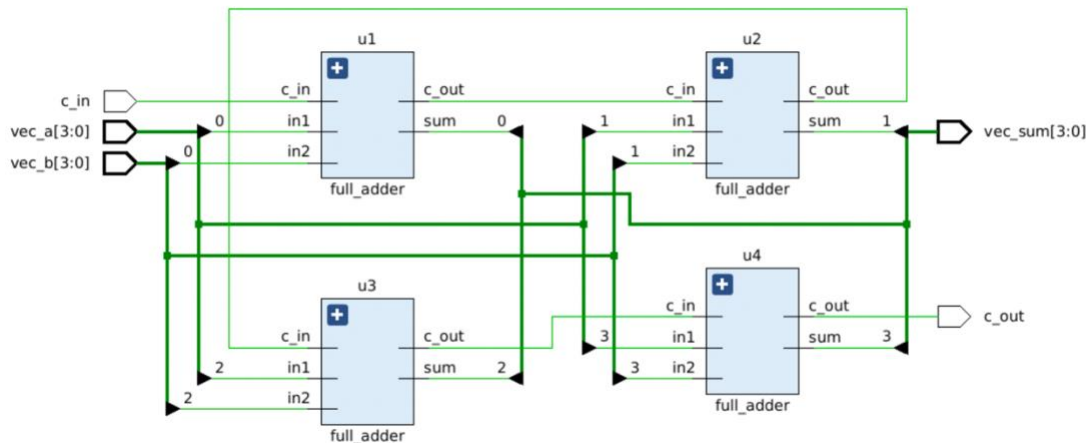


```

u3 : full_adder port map (in1 => vec_a(2), in2 => vec_b(2), c_in => c1, sum => vec_sum(2),
c_out => c2);
u4 : full_adder port map (in1 => vec_a(3), in2 => vec_b(3), c_in => c2, sum => vec_sum(3),
c_out => c_out);
end four_bit_adder_arch;

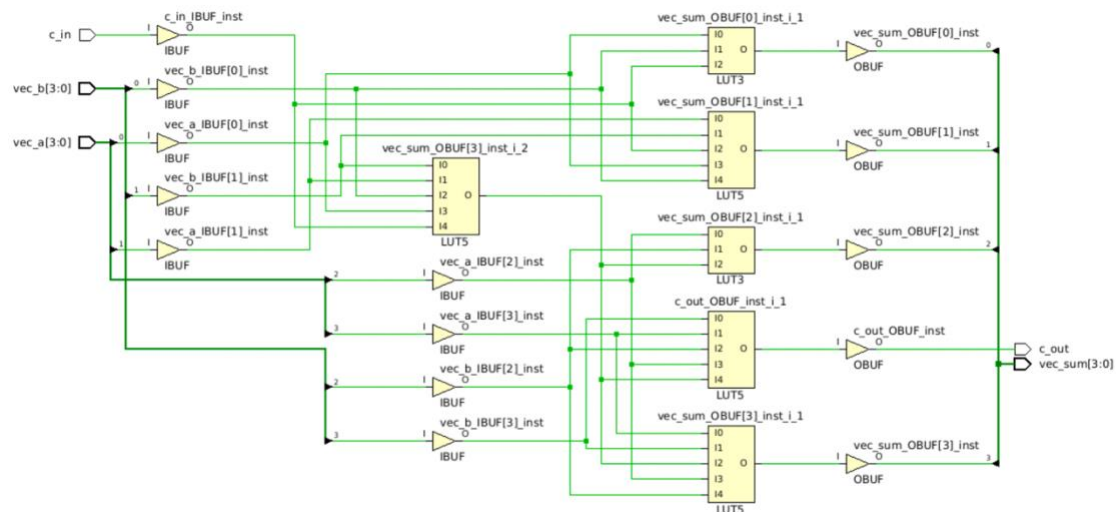
```

Άρα, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:

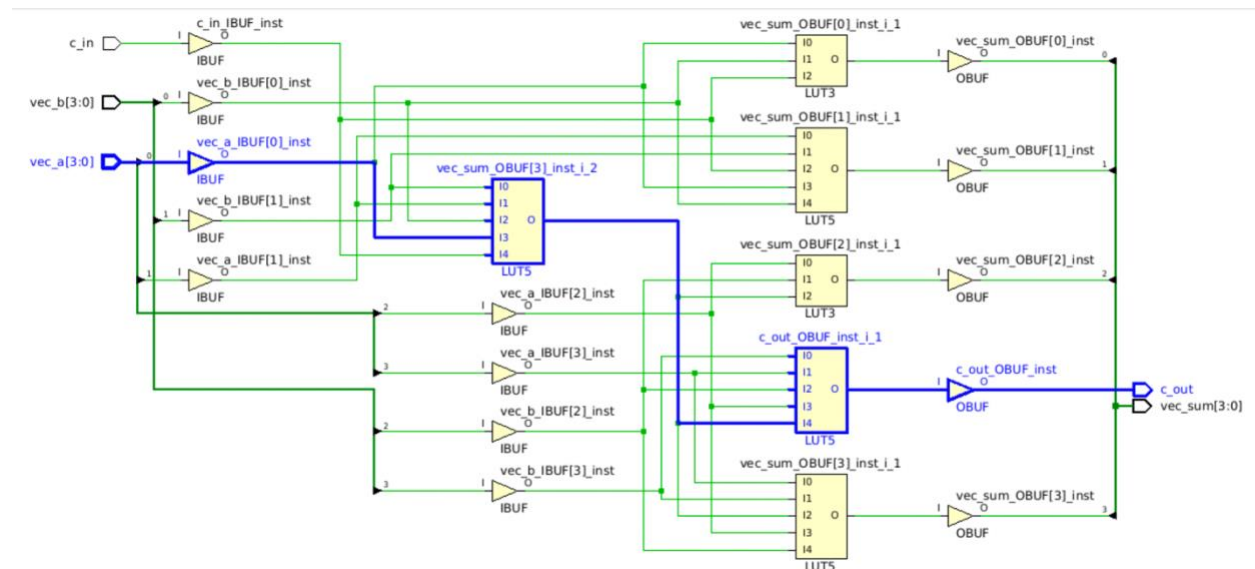


Το οποίο όντως αποτελείται από 4 Full Adders.

Τέλος με τη χρήση του synthesis λαμβάνουμε:



Όμοια με πριν, με τη βοήθεια του timing summary, επιθυμούμε να βρούμε το κρίσιμο μονοπάτι, όπως και την χρονική του καθυστέρηση.



Βλέπουμε ότι το κρίσιμο μονοπάτι, δηλαδή το μονοπάτι με τη μέγιστη καθυστέρηση, είναι αυτό από τη δεύτερη είσοδο μέχρι και το sum, με χρόνο 5.846 ns, όπως φαίνεται και παρακάτω:

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	4	5	3	vec_a[0]	c_out	5.846	3.780	2.066
Path 2	∞	4	5	3	vec_a[0]	vec_sum[2]	5.846	3.780	2.066
Path 3	∞	4	5	3	vec_a[0]	vec_sum[3]	5.838	3.772	2.066
Path 4	∞	3	4	3	c_in	vec_sum[0]	5.229	3.630	1.599
Path 5	∞	3	4	3	vec_b[0]	vec_sum[1]	5.229	3.630	1.599

Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity four_bit_adder_tb is
end four_bit_adder_tb;

architecture four_bit_adder_tb_arch of four_bit_adder_tb is

    component four_bit_adder is
        port (
            vec_a      : in std_logic_vector (4-1 downto 0);
            vec_b      : in std_logic_vector (4-1 downto 0);
            vec_sum     : out std_logic_vector (4-1 downto 0);
            c_in       : in std_logic;
            c_out      : out std_logic);
    end component;

    signal test_vec_a      : std_logic_vector (4-1 downto 0);

```

```

signal test_vec_b      : std_logic_vector (4-1 downto 0);
signal test_vec_sum    : std_logic_vector (4-1 downto 0);
signal test_c_in       : std_logic;
signal test_c_out      : std_logic;

begin
    uut : four_bit_adder port map (vec_a => test_vec_a, vec_b => test_vec_b, vec_sum =>
test_vec_sum, c_in => test_c_in, c_out => test_c_out);

    testing: process
    begin

        test_vec_a <= "0000";
        test_vec_b <= "0000";
        test_c_in <= '0';
        wait for 10 ns;

        test_vec_a <= "0001";
        test_vec_b <= "0010";
        test_c_in <= '0';
        wait for 10 ns;

        test_vec_a <= "1111";
        test_vec_b <= "1111";
        test_c_in <= '0';
        wait for 10 ns;

        test_vec_a <= "1010";
        test_vec_b <= "0101";
        test_c_in <= '1';
        wait for 10 ns;

        test_vec_a <= "0110";
        test_vec_b <= "0011";
        test_c_in <= '1';
        wait for 10 ns;

    end process;
end four_bit_adder_tb_arch;

```

Τρέχοντας την προσομοίωση λαμβάνουμε:



Προφανώς επειδή οι συνδυασμοί είναι πολλοί, χρησιμοποιήσαμε κάποια συγκεκριμένα σενάρια, με βάση τα οποία βλέπουμε ότι η υλοποίηση μας είναι σωστή.

4) Υλοποιήστε έναν BCD Πλήρη Αθροιστή (BCD Full Adder - BCD FA) με περιγραφή δομής (Structural). Να χρησιμοποιήσετε τη δομική μονάδα που υλοποιήθηκε στο Ερώτημα 3, οποιαδήποτε δομική μονάδα από τα προηγούμενα ερωτήματα, καθώς και επιπλέον λογική που θεωρείτε απαραίτητη.

Στη συνέχεια θέλουμε να φτιάξουμε έναν BCD πλήρη αθροιστή, ο οποίος στην έξοδο του θα εμφανίζει τις μονάδες του αποτελέσματος σε BCD μορφή (4 bits) και στο Output Carry θα δείχνει αν προκύπτει δεκάδα (carry προς το επόμενο ψηφίο). Έτσι, αν το αποτέλεσμα είναι μεγαλύτερο του 9 ή υπάρχει carry, θα γίνεται διόρθωση προσθέτοντας το 6, ώστε να πάρουμε τελικά τη σωστή BCD αναπαράσταση.

Η σχεδίαση του βασίζεται στον 4bit Binary Adder και αναπαρίσταται ως εξής:

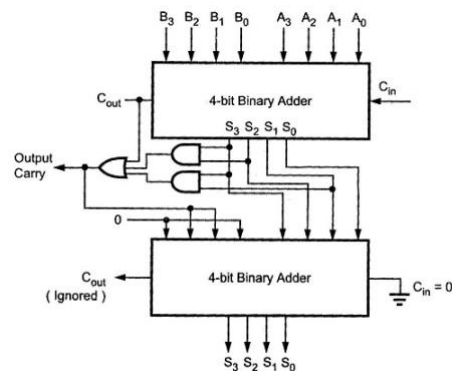


Fig. 3.32 Block diagram of BCD adder

Έτσι, με βάση το παραπάνω θα έχουμε:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bcd_bit_adder is
    port (
        vec_a      : in std_logic_vector (4-1 downto 0);
        vec_b      : in std_logic_vector (4-1 downto 0);
        vec_sum     : out std_logic_vector (4-1 downto 0);
        c_in       : in std_logic;
        c_out      : out std_logic);
end bcd_bit_adder;

architecture bcd_bit_adder_arch of bcd_bit_adder is
    signal sum1      : std_logic_vector (4-1 downto 0);
    signal c1       : std_logic;
```

```

signal c2          : std_logic;
signal and_top     : std_logic;
signal and_down    : std_logic;
signal or_gate     : std_logic;
signal test_vec_b2 : std_logic_vector (4-1 downto 0);

component four_bit_adder is
  port (
    vec_a      : in std_logic_vector (4-1 downto 0);
    vec_b      : in std_logic_vector (4-1 downto 0);
    vec_sum    : out std_logic_vector (4-1 downto 0);
    c_in       : in std_logic;
    c_out      : out std_logic);
end component;

begin
  u1 : four_bit_adder port map (vec_a => vec_a, vec_b => vec_b, vec_sum => sum1, c_in => c_in,
    c_out => c1);

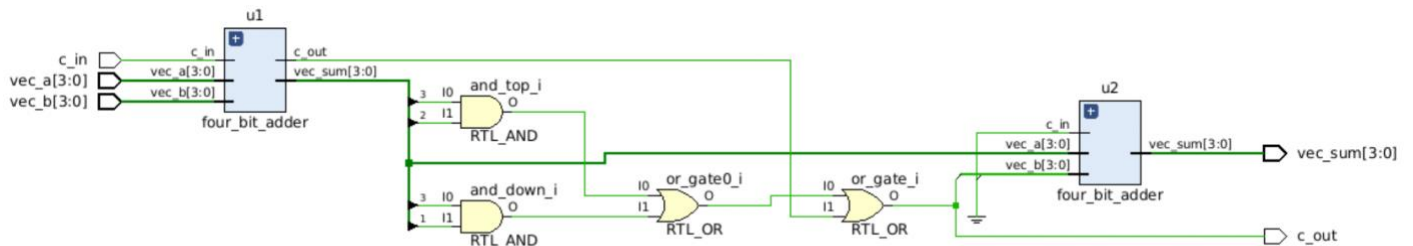
  and_top <= sum1(3) and sum1(2);
  and_down <= sum1(3) and sum1(1);
  or_gate <= and_top or and_down or c1;
  test_vec_b2 <= '0' & or_gate & or_gate & '0';

  u2 : four_bit_adder port map (vec_a => sum1, vec_b => test_vec_b2, vec_sum => vec_sum, c_in =>
    '0', c_out => c2);

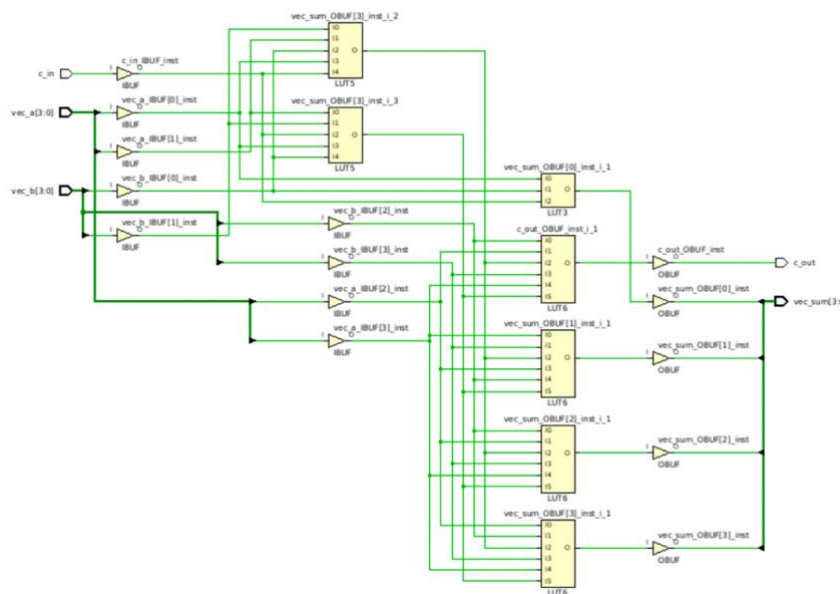
  c_out <= or_gate;
end bcd_bit_adder_arch;

```

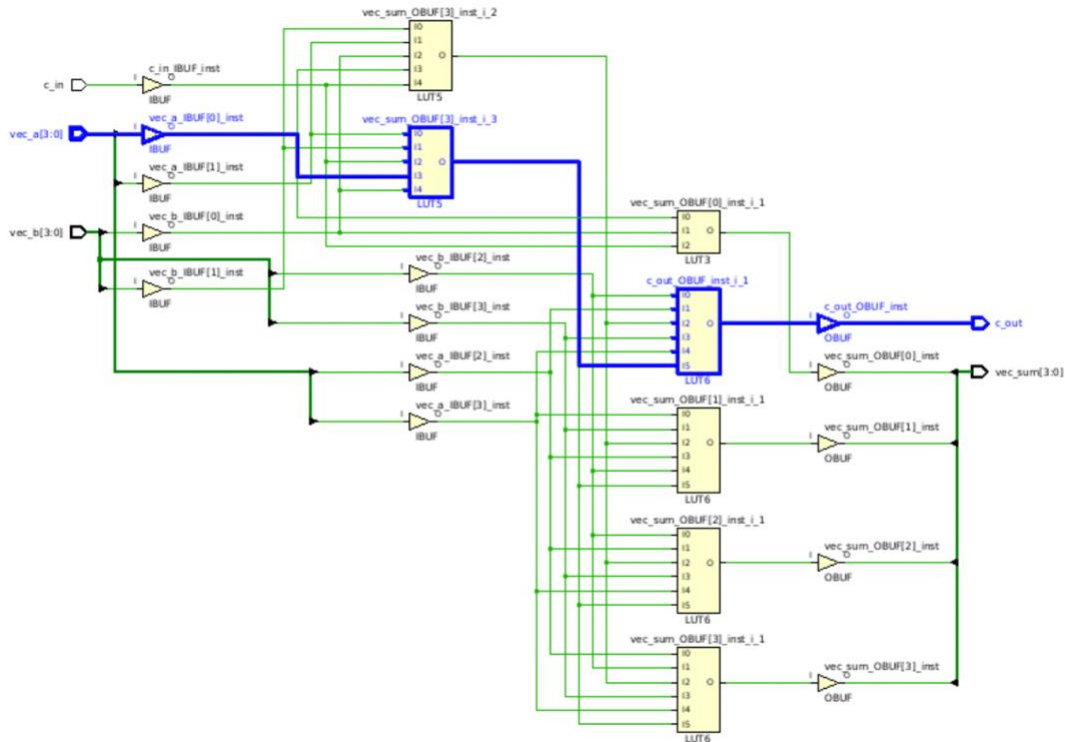
Άρα, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:



Τέλος με τη χρήση του synthesis λαμβάνουμε:



Όμοια με πριν, με τη βοήθεια του timing summary, επιθυμούμε να βρούμε το κρίσιμο μονοπάτι, όπως και την χρονική του καθυστέρηση.



Βλέπουμε ότι το κρίσιμο μονοπάτι, δηλαδή το μονοπάτι με τη μέγιστη καθυστέρηση, είναι αυτό από τη δεύτερη είσοδο μέχρι και το sum, με χρόνο 5.852 ns, όπως φαίνεται και παρακάτω:

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	4	5	4	vec_a[0]	c_out	5.852	3.780	2.072
Path 2	∞	4	5	4	vec_a[0]	vec_sum[1]	5.852	3.780	2.072
Path 3	∞	4	5	4	vec_a[0]	vec_sum[2]	5.852	3.780	2.072
Path 4	∞	4	5	4	vec_a[0]	vec_sum[3]	5.852	3.780	2.072
Path 5	∞	3	4	3	c_in	vec_sum[0]	5.229	3.630	1.599

Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bcd_bit_adder_tb is
end bcd_bit_adder_tb;

architecture bcd_bit_adder_tb_arch of bcd_bit_adder_tb is
  component bcd_bit_adder is
    port (
      vec_a      : in std_logic_vector (4-1 downto 0);
      vec_b      : in std_logic_vector (4-1 downto 0);
      vec_sum    : out std_logic_vector (4-1 downto 0);
    );
  end component;

```

```

        c_in      : in std_logic;
        c_out     : out std_logic);

end component;

signal test_vec_a      : std_logic_vector (4-1 downto 0);
signal test_vec_b      : std_logic_vector (4-1 downto 0);
signal test_vec_sum    : std_logic_vector (4-1 downto 0);
signal test_c_in       : std_logic;
signal test_c_out      : std_logic;

begin

    uut : bcd_bit_adder port map (vec_a => test_vec_a, vec_b => test_vec_b, vec_sum =>
test_vec_sum, c_in => test_c_in, c_out => test_c_out);

    testing: process
    begin
        test_vec_a <= "0000"; test_vec_b <= "0000"; test_c_in <= '0';
        wait for 10 ns;

        test_vec_a <= "0001"; test_vec_b <= "0001"; test_c_in <= '0';
        wait for 10 ns;

        test_vec_a <= "0011"; test_vec_b <= "0010"; test_c_in <= '1';
        wait for 10 ns;

        test_vec_a <= "1001"; test_vec_b <= "0001"; test_c_in <= '0';
        wait for 10 ns;

        test_vec_a <= "1001"; test_vec_b <= "0001"; test_c_in <= '1';
        wait for 10 ns;

        test_vec_a <= "0101"; test_vec_b <= "0101"; test_c_in <= '1';
        wait for 10 ns;

        test_vec_a <= "1001"; test_vec_b <= "1000"; test_c_in <= '1';
        wait for 10 ns;

        test_vec_a <= "1001"; test_vec_b <= "1001"; test_c_in <= '0';
        wait for 10 ns;

        test_vec_a <= "0001"; test_vec_b <= "1000"; test_c_in <= '0';
        wait for 10 ns;

        test_vec_a <= "0011"; test_vec_b <= "1001"; test_c_in <= '1';
        wait for 10 ns;

        test_vec_a <= "0101"; test_vec_b <= "0111"; test_c_in <= '1';
        wait for 10 ns;

    end process;
end bcd_bit_adder_tb_arch;

```

Τρέχοντας την προσομοίωση λαμβάνουμε:



Ο πίνακας αλήθειας για τα σενάρια που έχουμε δοκιμάσει είναι:

A (BCD)	B (BCD)	Cin	Δεκαδικό	Sum (BCD)	Cout
0000	0000	0	0+0+0=0	0000	0
0001	0001	0	1+1+0=2	0010	0
0011	0010	1	3+2+1=6	0110	0
1001	0001	0	9+1+0=10	0000	1
1001	0001	1	9+1+1=11	0001	1
0101	0101	1	5+5+1=11	0001	1
1001	1000	1	9+8+1=18	1000	1
1001	1001	0	9+9+0=18	1000	1
0001	1000	0	1+8+0=9	1001	0
0011	1001	1	3+9+1=13	0011	1
0101	0111	1	5+7+1=13	0011	1

5) Υλοποιήστε έναν Παράλληλο BCD Αθροιστή των 4 ψηφίων (4-BCD Parallel Adder - 4-BCD PA) με περιγραφή δομής (Structural), βασιζόμενοι στην δομική μονάδα του Ερωτήματος 4

Στο τελευταίο ερώτημα, θέλουμε να υλοποιήσουμε έναν παράλληλο BCD Αθροιστή των 4 ψηφίων, ακολουθώντας τη λογική του ερωτήματος 3, αλλά τώρα αντί για Full Adders, θα χρησιμοποιήσουμε BCD Adders.

Έτσι, με βάση το παραπάνω θα έχουμε:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bcd_four_bit_adder is
    port (
        vec_a      : in std_logic_vector (16-1 downto 0);
        vec_b      : in std_logic_vector (16-1 downto 0);
        vec_sum     : out std_logic_vector (16-1 downto 0);
        c_in       : in std_logic;
        c_out      : out std_logic);
end bcd_four_bit_adder;

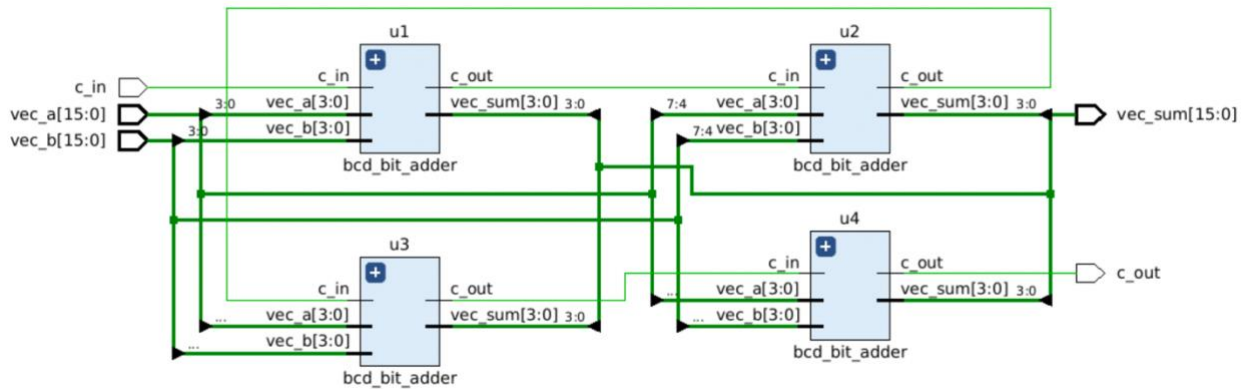
architecture bcd_four_bit_adder_arch of bcd_four_bit_adder is

    signal c_out1   : std_logic;
    signal c_out2   : std_logic;
    signal c_out3   : std_logic;

    component bcd_bit_adder is
        port (
            vec_a      : in std_logic_vector (4-1 downto 0);
            vec_b      : in std_logic_vector (4-1 downto 0);
            vec_sum     : out std_logic_vector (4-1 downto 0);
            c_in       : in std_logic;
            c_out      : out std_logic);
    end component;

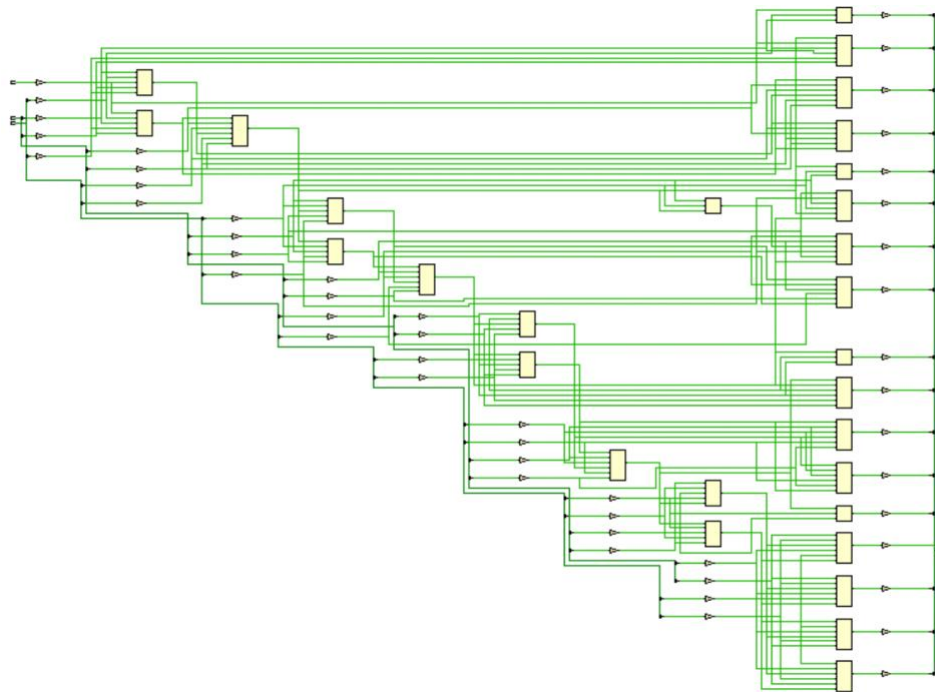
    begin
        u1 : bcd_bit_adder port map (vec_a => vec_a(3 downto 0), vec_b => vec_b(3 downto 0), vec_sum
=> vec_sum(3 downto 0), c_in => c_in, c_out => c_out1);
        u2 : bcd_bit_adder port map (vec_a => vec_a(7 downto 4), vec_b => vec_b(7 downto 4), vec_sum
=> vec_sum(7 downto 4), c_in => c_out1, c_out => c_out2);
        u3 : bcd_bit_adder port map (vec_a => vec_a(11 downto 8), vec_b => vec_b(11 downto 8),
vec_sum => vec_sum(11 downto 8), c_in => c_out2, c_out => c_out3);
        u4 : bcd_bit_adder port map (vec_a => vec_a(15 downto 12), vec_b => vec_b(15 downto 12),
vec_sum => vec_sum(15 downto 12), c_in => c_out3, c_out => c_out);
    end bcd_four_bit_adder_arch;
```

Άρα, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:

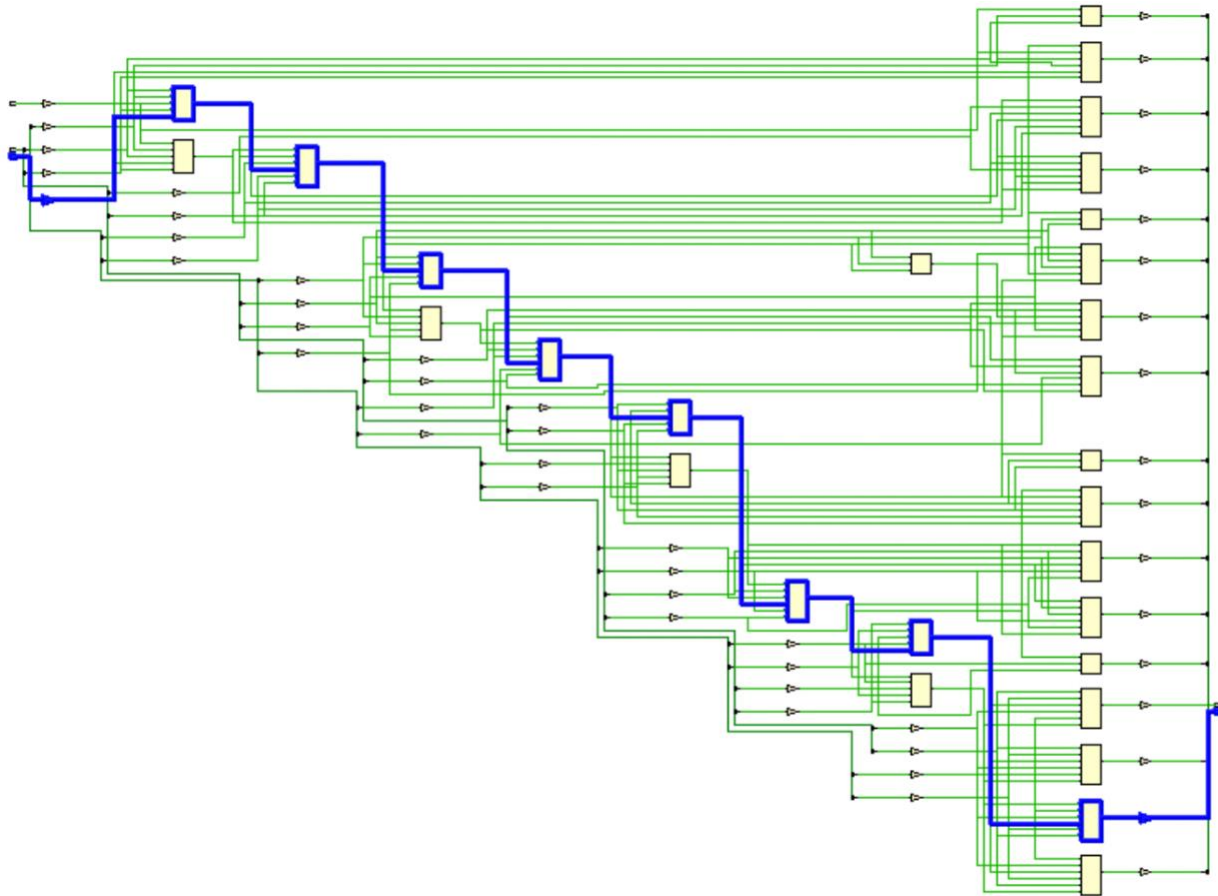


Το οποίο ορθά αποτελείται από τους 4 BCD αθροιστές.

Τέλος με τη χρήση του synthesis λαμβάνουμε:



Όμοια με πριν, με τη βοήθεια του timing summary, επιθυμούμε να βρούμε το κρίσιμο μονοπάτι, όπως και την χρονική του καθυστέρηση.



Βλέπουμε ότι το κρίσιμο μονοπάτι, δηλαδή το μονοπάτι με τη μέγιστη καθυστέρηση, είναι αυτό από τη δεύτερη είσοδο μέχρι και το sum, με χρόνο 9.399 ns, όπως φαίνεται και παρακάτω:

Name	Slack ^{^1}	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
↳ Path 1	∞	10	11	6	vec_b[1]	vec_sum[14]	9.399	4.498	4.901
↳ Path 2	∞	10	11	6	vec_b[1]	c_out	9.391	4.490	4.901
↳ Path 3	∞	10	11	6	vec_b[1]	vec_sum[13]	9.391	4.490	4.901
↳ Path 4	∞	10	11	6	vec_b[1]	vec_sum[15]	9.391	4.490	4.901
↳ Path 5	∞	9	10	6	vec_b[1]	vec_sum[12]	8.802	4.374	4.428
↳ Path 6	∞	9	10	6	vec_b[1]	vec_sum[9]	8.802	4.374	4.428
↳ Path 7	∞	8	9	6	vec_b[1]	vec_sum[10]	8.205	4.250	3.955

Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bcd_four_bit_adder_tb is
end bcd_four_bit_adder_tb;

architecture bcd_four_bit_adder_tb_arch of bcd_four_bit_adder_tb is

    component bcd_four_bit_adder is
        port (
            vec_a      : in std_logic_vector (16-1 downto 0);
            vec_b      : in std_logic_vector (16-1 downto 0);
            vec_sum     : out std_logic_vector (16-1 downto 0);
            c_in       : in std_logic;
            c_out      : out std_logic);
    end component;

    signal test_vec_a      : std_logic_vector (16-1 downto 0);
    signal test_vec_b      : std_logic_vector (16-1 downto 0);
    signal test_vec_sum    : std_logic_vector (16-1 downto 0);
    signal test_c_in      : std_logic;
    signal test_c_out     : std_logic;

    begin
        uut : bcd_four_bit_adder port map (vec_a => test_vec_a, vec_b => test_vec_b, vec_sum =>
test_vec_sum, c_in => test_c_in, c_out => test_c_out);

        testing: process

        begin
            test_vec_a <= "0000000000000000"; test_vec_b <= "0000000000000000"; test_c_in <= '0';
            wait for 100 ns;

            test_vec_a <= "0001001000110100"; test_vec_b <= "0010010001101001"; test_c_in <= '0';
            wait for 100 ns;

            test_vec_a <= "1001100110011001"; test_vec_b <= "0110011001100110"; test_c_in <= '1';
            wait for 100 ns;

            test_vec_a <= "1001100110011001"; test_vec_b <= "1001100110011001"; test_c_in <= '1';
            wait for 100 ns;

            test_vec_a <= "0000000000000000"; test_vec_b <= "1001100110011001"; test_c_in <= '0';
            wait for 100 ns;

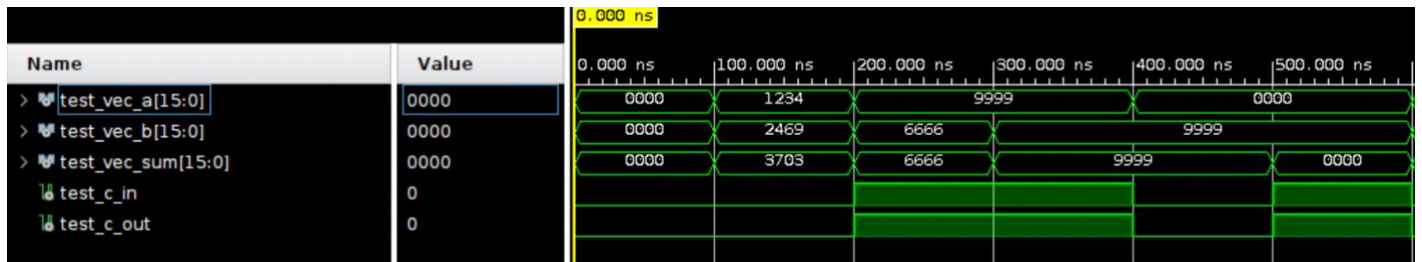
            test_vec_a <= "0000000000000000"; test_vec_b <= "1001100110011001"; test_c_in <= '1';
            wait for 100 ns;

            test_vec_a <= "1001100110011001"; test_vec_b <= "0000000000000000"; test_c_in <= '0';
            wait for 100 ns;

            test_vec_a <= "1001100110011001"; test_vec_b <= "0000000000000000"; test_c_in <= '1';
            wait for 100 ns;

        end process;
    end bcd_four_bit_adder_tb_arch;
```

Τρέχοντας την προσομοίωση λαμβάνουμε:



Τα αποτελέσματα της οποίας συνάδουν με τον παρακάτω πίνακα αλήθειας για τα σενάρια που χρησιμοποιήσαμε.

A (BCD)	B (BCD)	Cin	Sum (hex)	Cout
0000	0000	0	0000	0
1234	2469	0	3703	0
9999	6666	1	6666	1
9999	9999	1	9999	1
0000	9999	0	9999	0
0000	9999	1	0000	1